- **String (Checking Content):**

  - *.isalpha()* → True if all characters are alphabets.
  - *.isdigit()* → True if all characters are digits.
  - *.isalnum()* → True if all characters are either letters (a-z,A-Z) or digits(0-9).

    Returns False if string contains non-alphanumeric characters (i.e. spaces, punctuation marks, special symbols), or if string is empty.
  - *.islower()* → True if all characters are lowercase.
  - *.isupper()* → True if all characters are uppercase.
  - *.istitle()* → True if string is in title case.

Method	Accepts	Example "123"	"²" (superscript)	"½" (fraction)	"四" (Chinese 4)
isdecimal()	Only 0–9	✅ True	❌ False	❌ False	❌ False
isdigit()	Digits + superscripts/subscripts	✅ True	✅ True	❌ False	❌ False
isnumeric()	Digits + superscripts + fractions + other numerals	✅ True	✅ True	✅ True	✅ True

- **String Manipulation (Tasks)**

1. Write a program to get string from user and reverse it.
2. Write a program to get string from user and count vowels in it.
3. Write a program to check if the given string is palindrome.
4. Write a program to remove spaces from string.

   string = “ Machine Learning ”
5. Write a program to find frequency of each character in a given string.

Files:

A file is a sequence of bytes stored on a secondary memory device to which Python can interact with. This interaction is known as file handling.

Each byte in a file can be thought as integer with value between 0 – 255.

File handling involves operations *such as creating, opening, reading from, writing to and closing files*.

File I/O: Reading data from file into Python is termed as *input operation*, writing data from Python to a file is termed as *output operation*.

Files are of various types:

- Text file *(.txt, .csv, .docx)*
- Binary file *(.bin, .exe, .png, .mp3)*
- Spreadsheet *(.xlsx)*

▪ File opening:

open() function is used to open a file. It returns a file object, which is used to perform operations on a file.

Syntax: *file_object = open(filename, mode)*

Modes:

'r' (Read)	Default mode. Open a file for reading, raise an error if file does not exist.
'w' (Write)	Opens a file for writing. Creates a file if it doesn't exist; otherwise it erase the existing contents.
'a' (Append)	Opens a file for appending, Creates a file if it doesn't exist; otherwise new data is added to the end of file.
'x' (Exclusive Creation)	Creates a new file. Raises an error if the file already exist.
'b' (Binary)	Used in conjunction with other modes such as ***'rb'***, ***'wb'***. For handling binary files like image or audio.
'r+' (Read and write)	Allows you to read and write without erasing whole file content (file must exist)
'w+' (write and read)	Allows you to write and read, If file exists the entire file is truncated empty
'a+' (append and read)	Write data to the end of file and read existing data from file. (pointer on end)
't' (Text)	Default mode to text files, handling content as strings.

- **File close:**

close() function is used to close a file.

Syntax:

file_object.close()

- **File read:**

1. Read the entire content or ‘n’ characters of the file.

   file_content = file_object.read() *or*

   file_content = file_object.read(n)

2. Read a single line from the file

   file_content = file_object.readline()

3. Read all lines into a list

   file_content = file_object.readlines()

- **File Write:**

1. Write a string to a file.

   file_object.write(string)

2. Write a list of strings to a file.

   file_object.writelines(list_of_strings)

- **File Opening (Recommended Method):**

The *with* statement is used for resource management and exception handling, providing a clean and efficient way to ensure that resources are properly acquired and released.

Primary benefit of *with* statement is that it guarantees resource cleanup. You do not need to explicitly call *close()*, as the context manager handles this automatically through its *__exit__()* method.

Syntax:

```
with open(file_path, mode) as f:
    content = f.read()
    print(content)
```

- **File Data Manipulation:**

After reading data from a file, Python's built-in data structures and functions can be used for manipulation.

1. Text data: Strings can be split, joined, searched, replaced and formatted.
2. Structured data (CSV, JSON):

```
file_object = open('C:/Users/ABC/Desktop/Saylani AI and data science course/f
ile_1.txt')
file_object
<_io.TextIOWrapper name='C:/Users/ABC/Desktop/Saylani AI and data science cou
rse/file_1.txt' mode='r' encoding='cp1252'>
file_object.read()
'First line\nSecond line\nThird line\nFourth line'
file_object.read(10)
''
file_object.read()
''
file_object.close()

file_object = open('C:/Users/ABC/Desktop/Saylani AI and data science course/f
ile_1.txt')
print(file_object.read())
First line
Second line
Third line
Fourth line
file_object.close()
```

```
file_object = open('C:/Users/ABC/Desktop/Saylani AI and data science course/f
ile_1.txt')
print(file_object.read(10))
First line
print(file_object.read(10))

Second li
print(file_object.read(10))
ne
Third l
file_object.close()
```

```
file_object = open('C:/Users/ABC/Desktop/Saylani AI and data science course/f
ile_1.txt')
file_object.readline()
'First line\n'
file_object.readline()
'Second line\n'
file_object.readline()
'Third line\n'
file_object.readline()
'Fourth line'
file_object.readline()
''

file_object.close()

file_object = open('C:/Users/ABC/Desktop/Saylani AI and data science course/f
ile_1.txt')
file_object.readline(2)
'Fi'
file_object.readline(8)
'rst line'
file_object.readline(1)
'\n'
file_object.readline(12)
'Second line\n'
file_object.readline(6)
'Third '
file_object.close()
```

```
file_object = open('C:/Users/ABC/Desktop/Saylani AI and data science course/f
ile_1.txt')
file_object.readlines()
['First line\n', 'Second line\n', 'Third line\n', 'Fourth line']
file_object.readlines()
[]
file_object.close()

file_object = open('C:/Users/ABC/Desktop/Saylani AI and data science course/f
ile_1.txt')
p = file_object.readlines()
p
['First line\n', 'Second line\n', 'Third line\n', 'Fourth line']
print(p[2])
Third line

file_object.close()
```

```
file_path = 'C:/Users/ABC/Desktop/Saylani AI and data science course/file_1.t
xt'

with open(file_path) as f:
    print(f.readline())


First line

f.readline()
Traceback (most recent call last):
  File "<pyshell#1103>", line 1, in <module>
    f.readline()
ValueError: I/O operation on closed file.

with open(file_path) as f:
    print(f.readlines())


['First line\n', 'Second line\n', 'Third line\n', 'Fourth line']
```